UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE · LEZIONE 7

Web service SOAP in Java

Laboratorio: scriviamo un servizio SOAP vero con Maven e Jakarta XML Web Services, leggiamo il WSDL che genera e costruiamo il client partendo solo dal contratto.

5G TPSI Maven, @WebService, Endpoint, WSDL, wsimport, Fault

PER COMINCIARE

Che rapporto hai con Maven?

SONDAGGIO

- **A** Mai sentito
- **B** So che cos'è, non l'ho mai usato
- **C** L'ho usato in un progetto
- **D** Lo uso spesso

IL PIANO DEL LABORATORIO

Dal codice al contratto, dal contratto al client

1 · Servizio

Una classe Java con `@WebService`

2 · Pubblica

`Endpoint.publish` su `localhost:8080/calc`

3 · WSDL

Generato da solo: `?wsdl`

4 · Client

`wsimport` legge il WSDL e genera lo stub

Code-first

Scrivo il codice, il framework genera il WSDL. È quello che facciamo per il servizio.

Contract-first

Parto dal WSDL concordato e genero il codice. È quello che facciamo per il client, e che si fa tra aziende.

METTITI ALLA PROVA

Copi un esempio del 2015 con `import javax.jws.WebService;` e con JDK 21 non compila. Perché?

QUIZ

A Bisogna attivare un'opzione del compilatore per SOAP

B Il pacchetto è stato rinominato `java.soap`

C JAX-WS è uscito dal JDK con Java 11: ora è una libreria, Jakarta XML Web Services

D Java 21 non supporta più i web service SOAP

AL COMPUTER · 15 minuti

Laboratorio 1: il progetto Maven del servizio

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>it.scuola</groupId>
  <artifactId>calc-soap</artifactId>
  <version>1.0</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.sun.xml.ws</groupId>
      <artifactId>jaxws-rt</artifactId>
      <version>4.0.3</version>
    </dependency>
  </dependencies>
</project>
```

1. IntelliJ: **New Project**, build system **Maven**, JDK 21, nome **calc-soap**, GroupId **it.scuola**.
2. Sostituisci il **pom.xml** con questo.
3. Premi l'icona **Load Maven Changes** (la "m" con le frecce).
4. Crea il pacchetto `it.scuola.calc` in `src/main/java`.

Controllo: in **External Libraries** compaiono `jaxws-rt-4.0.3` e altre librerie scaricate da sole (jaxb, saaj…). Nessuna riga rossa nel pom.

AL COMPUTER · 15 minuti

Laboratorio 2: la classe del servizio

```java
package it.scuola.calc;

import jakarta.jws.WebMethod;
import jakarta.jws.WebParam;
import jakarta.jws.WebService;

@WebService(serviceName = "CalcolatriceService")
public class Calcolatrice {
  @WebMethod
  public int somma(@WebParam(name = "a") int a,
                   @WebParam(name = "b") int b) {
    return a + b;
  }
  @WebMethod
  public double dividi(@WebParam(name = "a") double a,
                       @WebParam(name = "b") double b) {
    if (b == 0) throw new IllegalArgumentException("Divisione per zero");
    return a / b;
  }
}
```

Annotazione	Effetto
`@WebService`	La classe diventa un servizio; `serviceName` dà il nome nel WSDL
`@WebMethod`	Il metodo diventa un'operazione
`@WebParam`	Nome dell'elemento XML del parametro

AL COMPUTER · 15 minuti

Laboratorio 3: pubblica e leggi il WSDL

```java
package it.scuola.calc;

import jakarta.xml.ws.Endpoint;

public class Server {
  public static void main(String[] args) {
    String url = "http://localhost:8080/calc";
    Endpoint.publish(url, new Calcolatrice());
    System.out.println("Servizio attivo: " + url + "?wsdl");
  }
}
```

Avvialo con il pulsante **Run** dell'IDE accanto al `main`.

1. Crea **Server.java** ed eseguilo.
2. Apri nel browser `http://localhost:8080/calc?wsdl`
3. Apri anche `http://localhost:8080/calc` e `?xsd=1`.

Controllo: un WSDL con `name="CalcolatriceService"` e `targetNamespace="http://calc.scuola.it/"`, operazioni **somma** e **dividi**. Senza ?wsdl: una pagina "Web Services" con l'elenco degli endpoint.

PUZZLE

Dal Java al WSDL: chi diventa che cosa?

Pacchetto it.scuola.calc	---	targetNamespace http://calc.scuola.it/
Classe Calcolatrice	---	elemento a di tipo xs:int nello schema
serviceName	---	portType Calcolatrice
Metodo somma	---	operation somma, messaggi somma e sommaResponse
@WebParam(name = "a") int	---	service CalcolatriceService
Valore restituito	---	elemento return

AL COMPUTER · 15 minuti

Laboratorio 4: chiamalo con curl, come nella lezione 6

```
curl -i -H "Content-Type: text/xml; charset=utf-8"
     --data-binary @somma.xml http://localhost:8080/calc
```

1. Riusa **somma.xml** della lezione 6 (o riscrivilo): stessa busta, stesso namespace.
2. Crea **dividi.xml**: operazione `c:dividi` con a = 20 e b = 0. Invialo.
3. Rimanda somma.xml con `Content-Type: application/json`.
4. Rinomina l'operazione in `c:moltiplica` e invia.
5. Copia le quattro risposte in **soap-java-curl-cognome.txt**.

Controllo:

1. **200**, `<return>42</return>`
2. **500**, Fault `S:Server`, "Divisione per zero"
3. **415** Unsupported Media Type
4. **500**, Fault `S:Client`, "Cannot find dispatch method for {http://calc.scuola.it/}moltiplica"

QUIZ A TRABOCCHETTO

Mandi al servizio vero la busta senza l'elemento `<a>`. Che cosa risponde?

QUIZ

- **A** 200 con `<return>22</return>`
- **B** Un Fault Client: "manca il parametro a"
- **C** 400 Bad Request
- **D** Il server va in crash e si ferma

AL COMPUTER · 20 minuti

Laboratorio 5: il client nasce dal WSDL

```
<!-- pom.xml di calc-client: stessa dipendenza jaxws-rt, più -->
<build><plugins>
  <plugin>
    <groupId>com.sun.xml.ws</groupId>
    <artifactId>jaxws-maven-plugin</artifactId>
    <version>4.0.3</version>
    <executions><execution>
      <goals><goal>wsimport</goal></goals>
      <configuration>
        <wsdlUrls>
          <wsdlUrl>http://localhost:8080/calc?wsdl</wsdlUrl>
        </wsdlUrls>
        <packageName>it.scuola.client.gen</packageName>
      </configuration>
    </execution></executions>
  </plugin>
</plugins></build>
```

1. Lascia acceso il servizio. Crea un secondo progetto Maven **calc-client**.
2. Nel pom: la stessa dipendenza `jaxws-rt` e, dopo `</dependencies>`, questo blocco `build`.
3. Esegui **mvn compile** (in IntelliJ: finestra Maven, Lifecycle, compile).

Controllo: nel log "parsing WSDL… Generating code…" e in `target/generated-sources/wsimport` le classi **Calcolatrice**, **CalcolatriceService**, **Somma**, **SommaResponse**, **ObjectFactory**…

AL COMPUTER · 15 minuti

Laboratorio 6: il client chiama, e gestisce il Fault

```
package it.scuola.client;

import it.scuola.client.gen.Calcolatrice;
import it.scuola.client.gen.CalcolatriceService;

public class Client {
  public static void main(String[] args) {
    Calcolatrice calc =
        new CalcolatriceService().getCalcolatricePort();
    System.out.println("20 + 22 = " + calc.somma(20, 22));
    try {
      System.out.println(calc.dividi(1, 0));
    } catch (jakarta.xml.ws.soap.SOAPFaultException e) {
      System.out.println("Fault dal server: " + e.getMessage());
    }
  }
}
```

1. Crea **Client.java** nel pacchetto `it.scuola.client` ed eseguilo.
2. Spegni il servizio e rilancia il client: che cosa succede?

Controllo: **20 + 22 = 42**, poi **Fault dal server: Client received SOAP Fault from server: Divisione per zero**… A servizio spento: un'eccezione di connessione.

QUANDO NON FUNZIONA

Gli errori che incontrerai, e come uscirne

Sintomo	Causa probabile	Rimedio
`BindException: Address already in use`	La porta 8080 è già occupata	Ferma l'altra esecuzione o cambia porta
`package jakarta.jws does not exist`	Dipendenza non caricata	Load Maven Changes, controlla il pom e il proxy
`wsimport: Connection refused`	Servizio spento mentre generi il client	Avvia Server, poi **mvn compile**
`415 Unsupported Media Type`	Content-Type sbagliato	`text/xml` per SOAP 1.1
`Cannot find dispatch method`	Operazione o namespace sbagliati	Confronta la busta con il WSDL
`javax.jws non trovato`	Esempio scritto per Java 8	Sostituisci `javax` con `jakarta`

MENTI CHE CI HANNO PRECEDUTO

Meno chiacchiere

> **Parlare costa poco. Fammi vedere il codice.**

Linus Torvalds, creatore di Linux, in un messaggio alla mailing list del kernel Linux, 25 agosto 2000



AL COMPUTER · 40 minuti · lavoro di gruppo

Sfida a squadre: il servizio Registro

1. Squadre da 3: uno scrive il servizio, uno il client, uno prepara le buste di test con curl.
2. Aggiungi al progetto un servizio **Registro** con `aggiungiVoto(studente, voto)` e `media(studente)`, con i voti in una `Map`.
3. Voto fuori da 1–10 o studente sconosciuto: lancia un'eccezione con un messaggio chiaro.
4. Pubblicalo su `/registro` nello stesso `main`, rigenera il client con wsimport e provalo.
5. Consegna **registro-soap-squadra.zip** (i due progetti senza la cartella `target`, più le buste di test).

40:00

Controllo: dopo aggiungiVoto("Ada", 8) e aggiungiVoto("Ada", 9), media("Ada") vale **8.5**; aggiungiVoto("Ada", 11) restituisce un **Fault**.

OGGI

Un vecchio servizio SOAP, un nuovo utente: l'AI



Il ponte, in tre livelli

1. Agente AI → MCP (JSON-RPC)
2. Server MCP in Java → stub wsimport
3. Stub → servizio SOAP (XML su HTTP)

BIGLIETTO D'USCITA

Completa prima di uscire

Il file che descrive un progetto Maven e le sue dipendenze si chiama _____.

L'annotazione che trasforma una classe in un servizio SOAP è _____.

Il metodo che pubblica il servizio senza un application server è _____.

Lo strumento che genera il client dal WSDL si chiama _____.

Lato client un Fault arriva come _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- JAX-WS non è più nel JDK: Jakarta XML Web Services con Maven
- @WebService, @WebMethod, @WebParam, Endpoint.publish
- Code-first per il servizio, il WSDL si genera da solo
- wsimport: dal WSDL allo stub, contract-first
- Eccezione nel servizio = Fault; nel client SOAPFaultException
- Senza validazione un parametro mancante diventa 0

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.